Step 1

Team Member A

In [6]:
```python
import numpy as np
import pandas as pd

# --- 1. Model and Simulation Parameters ---
# General Parameters
S0 = 80.0
K = 80.0  # ATM Strike
r = 0.055
T = 0.25  # 3 months

# Heston Model Specific Parameters
v0 = 0.032      # Initial variance
kappa = 1.85    # Speed of mean reversion
theta = 0.045   # Long-run variance
xi = 0.60       # Volatility of volatility (assumed)

# Monte Carlo Simulation Controls
N_SIMS = 60000
N_STEPS = 250
BUMP_SIZE = 0.8 # 1% of S0 for Greek calculation

# --- 2. Function Definitions ---

def price_heston_mc(s0_val, k_val, rho, seed):
    """
    Prices European Call and Put options using the Heston model via Monte Carlo.

    Args:
        s0_val (float): Initial stock price for this run.
        k_val (float): Strike price.
        rho (float): Correlation parameter.
        seed (int): Seed for the random number generator for reproducibility.

    Returns:
        tuple: (call_price, put_price, call_standard_error, put_standard_error)
    """
    dt = T / N_STEPS
    sqrt_dt = np.sqrt(dt)

    # Use a dedicated random number generator for consistency
    rng = np.random.default_rng(seed)

    # Pre-allocate arrays for paths
    S = np.full(N_SIMS, s0_val, dtype=np.float64)
    v = np.full(N_SIMS, v0, dtype=np.float64)

    for _ in range(N_STEPS):
        # Generate correlated random shocks
        z_v = rng.standard_normal(N_SIMS)
        z_s_ind = rng.standard_normal(N_SIMS)
        z_s = rho * z_v + np.sqrt(1 - rho**2) * z_s_ind

        # Update variance using the full truncation scheme
        v_pos = np.maximum(v, 0)
        v = v + kappa * (theta - v_pos) * dt + xi * np.sqrt(v_pos) * sqrt_dt * z_v

        # Update stock price
        S = S * np.exp((r - 0.5 * np.maximum(v, 0)) * dt + np.sqrt(np.maximum(v, 0)) * sqrt_dt

    # Calculate discounted payoffs and prices
    discount_factor = np.exp(-r * T)
    call_payoffs = np.maximum(S - k_val, 0)
    put_payoffs = np.maximum(k_val - S, 0)
```

```
    call_price = np.mean(call_payoffs) * discount_factor
    put_price = np.mean(put_payoffs) * discount_factor

    # Calculate standard errors
    call_se = (np.std(call_payoffs, ddof=1) / np.sqrt(N_SIMS)) * discount_factor
    put_se = (np.std(put_payoffs, ddof=1) / np.sqrt(N_SIMS)) * discount_factor

    return call_price, put_price, call_se, put_se

def calculate_heston_greeks(s0_val, k_val, rho, h, base_seed):
    """
    Calculates Delta and Gamma for Call and Put options using Common Random Numbers.
    """
    # Price at S0, S0+h, and S0-h using the SAME random seed for all three calls
    c_mid, p_mid, _, _ = price_heston_mc(s0_val, k_val, rho, seed=base_seed)
    c_up, p_up, _, _ = price_heston_mc(s0_val + h, k_val, rho, seed=base_seed)
    c_down, p_down, _, _ = price_heston_mc(s0_val - h, k_val, rho, seed=base_seed)

    # Central difference for Delta
    delta_call = (c_up - c_down) / (2 * h)
    delta_put = (p_up - p_down) / (2 * h)

    # Central difference for Gamma
    gamma_call = (c_up - 2 * c_mid + c_down) / (h**2)
    gamma_put = (p_up - 2 * p_mid + p_down) / (h**2)

    return delta_call, gamma_call, delta_put, gamma_put

# --- 3. Main Execution Block ---

# Q5: Price options for rho = -0.30
rho_q5 = -0.30
c5, p5, c5_se, p5_se = price_heston_mc(S0, K, rho_q5, seed=42)

# Q7: Calculate Greeks for rho = -0.30
delta_c5, gamma_c5, delta_p5, gamma_p5 = calculate_heston_greeks(S0, K, rho_q5, h=BUMP_SIZE, b
```

```
# Q6: Price options for rho = -0.70
rho_q6 = -0.70
c6, p6, c6_se, p6_se = price_heston_mc(S0, K, rho_q6, seed=42)

# Q7: Calculate Greeks for rho = -0.70
delta_c6, gamma_c6, delta_p6, gamma_p6 = calculate_heston_greeks(S0, K, rho_q6, h=BUMP_SIZE, b

# --- 4. Format and Display Results ---

results_data = {
    'Metric': ['Call Price', 'Call SE', 'Put Price', 'Put SE', 'Delta Call (Δ)', 'Gamma Call (
    'Q5 (ρ=-0.30)': [c5, c5_se, p5, p5_se, delta_c5, gamma_c5, delta_p5, gamma_p5],
    'Q6 (ρ=-0.70)': [c6, c6_se, p6, p6_se, delta_c6, gamma_c6, delta_p6, gamma_p6]
}
results_df_A = pd.DataFrame(results_data).set_index('Metric')

print("--- Team Member A: Heston Model Results ---")
print(results_df_A.round(4))
```

```
--- Team Member A: Heston Model Results ---
                Q5 (ρ=-0.30)  Q6 (ρ=-0.70)
Metric
Call Price            2.3657        1.1838
Call SE               0.0160        0.0090
Put Price             3.0094        4.1131
Put SE                0.0191        0.0230
Delta Call (Δ)        0.5055        0.3837
Gamma Call (Γ)        0.0677        0.0747
Delta Put (Δ)        -0.4730       -0.5661
Gamma Put (Γ)         0.0677        0.0747
```

Team Member B

In [7]:
```
import numpy as np
import pandas as pd
```

```python
# --- 1. Model and Simulation Parameters ---
# General Parameters
S0 = 80.0
K = 80.0  # ATM Strike
r = 0.055
T = 0.25  # 3 months
sigma = 0.35

# Merton Model Specific Parameters
mu_j = -0.5
delta_j = 0.22

# Monte Carlo Simulation Controls
N_SIMS = 100000  # Using a higher number for stable Greek estimates
N_STEPS = 500
BUMP_SIZE = 1.0

# --- 2. Function Definitions ---

def simulate_merton_growth_factors(lam, seed):
    """
    Simulates the final growth factors (ST/S0) for the Merton model.
    This is the core computational engine.

    Args:
        lam (float): Jump intensity parameter.
        seed (int): Seed for the random number generator.

    Returns:
        np.array: Array of simulated growth factors.
    """
    dt = T / N_STEPS
    sqrt_dt = np.sqrt(dt)
    rng = np.random.default_rng(seed)

    # Drift adjustment for the jump component
    k = np.exp(mu_j + 0.5 * delta_j**2) - 1.0
    drift = (r - lam * k - 0.5 * sigma**2) * dt

    # Generate all random numbers needed upfront
    z = rng.standard_normal(size=(N_STEPS, N_SIMS))
    z_j = rng.standard_normal(size=(N_STEPS, N_SIMS))
    poisson_jumps = rng.poisson(lam * dt, size=(N_STEPS, N_SIMS))

    # Calculate the sum of log returns over all steps
    log_returns = np.sum(drift + sigma * sqrt_dt * z + mu_j * poisson_jumps + delta_j * np.sqr

    return np.exp(log_returns)

def price_merton_options_and_greeks(lam, seed):
    """
    Prices options and calculates Greeks efficiently using simulated growth factors.
    """
    discount_factor = np.exp(-r * T)

    # Step 1: Simulate the growth factors ONCE
    growth_factors = simulate_merton_growth_factors(lam, seed=seed)

    # Step 2: Calculate prices for S0, S0+h, S0-h using the SAME growth factors
    # Payoffs at S0
    st_mid = S0 * growth_factors
    call_payoffs_mid = np.maximum(st_mid - K, 0)
    put_payoffs_mid = np.maximum(K - st_mid, 0)

    # Payoffs at S0 + h
    st_up = (S0 + BUMP_SIZE) * growth_factors
    call_payoffs_up = np.maximum(st_up - K, 0)
    put_payoffs_up = np.maximum(K - st_up, 0)

    # Payoffs at S0 - h
    st_down = (S0 - BUMP_SIZE) * growth_factors
    call_payoffs_down = np.maximum(st_down - K, 0)
```

```
    put_payoffs_down = np.maximum(K - st_down, 0)

    # Step 3: Calculate final prices and standard errors
    call_price = np.mean(call_payoffs_mid) * discount_factor
    put_price = np.mean(put_payoffs_mid) * discount_factor
    call_se = (np.std(call_payoffs_mid, ddof=1) / np.sqrt(N_SIMS)) * discount_factor
    put_se = (np.std(put_payoffs_mid, ddof=1) / np.sqrt(N_SIMS)) * discount_factor

    # Step 4: Calculate Greeks
    c_up = np.mean(call_payoffs_up) * discount_factor
    c_down = np.mean(call_payoffs_down) * discount_factor
    p_up = np.mean(put_payoffs_up) * discount_factor
    p_down = np.mean(put_payoffs_down) * discount_factor

    delta_call = (c_up - c_down) / (2 * BUMP_SIZE)
    gamma_call = (c_up - 2 * call_price + c_down) / (BUMP_SIZE**2)
    delta_put = (p_up - p_down) / (2 * BUMP_SIZE)
    gamma_put = (p_up - 2 * put_price + p_down) / (BUMP_SIZE**2)

    return {
        'Call Price': call_price, 'Call SE': call_se, 'Put Price': put_price, 'Put SE': put_se
        'Delta Call (Δ)': delta_call, 'Gamma Call (Γ)': gamma_call,
        'Delta Put (Δ)': delta_put, 'Gamma Put (Γ)': gamma_put
    }

# --- 3. Main Execution Block ---

# Q8 & Q10: Price options and Greeks for lambda = 0.75
results_q8 = price_merton_options_and_greeks(lam=0.75, seed=2025)

# Q9 & Q10: Price options and Greeks for lambda = 0.25
results_q9 = price_merton_options_and_greeks(lam=0.25, seed=2025)

# --- 4. Format and Display Results ---

results_df_B = pd.DataFrame([results_q8, results_q9], index=['Q8 (λ=0.75)', 'Q9 (λ=0.25)']).T

print("--- Team Member B: Merton Model Results ---")
print(results_df_B.round(4))
```

```
--- Team Member B: Merton Model Results ---
                Q8 (λ=0.75)  Q9 (λ=0.25)
Call Price           8.3492       6.8708
Call SE              0.0364       0.0321
Put Price            7.1973       5.7659
Put SE               0.0388       0.0295
Delta Call (Δ)       0.6495       0.5992
Gamma Call (Γ)       0.0219       0.0265
Delta Put (Δ)       -0.3512      -0.4010
Gamma Put (Γ)        0.0219       0.0265
```

Team Member C

Q11: Put-Call Parity Verification

In [8]:

```
import numpy as np
import pandas as pd

# --- 1. General Parameters and Results from Team A & B ---
S0 = 80.0
K = 80.0
r = 0.055
T = 0.25

# Manually input the results from the refined scripts for a combined analysis
data = {
    'Model': ['Heston (ρ=-0.30)', 'Heston (ρ=-0.70)', 'Merton (λ=0.75)', 'Merton (λ=0.25)'],
    'Call Price (C)': [2.3657, 1.1838, 8.3093, 6.8856],
    'Put Price (P)': [3.0094, 4.1131, 7.1932, 5.7441],
    'Call SE': [0.0160, 0.0090, 0.0298, 0.0276],
    'Put SE': [0.0191, 0.0230, 0.0275, 0.0278]
```

```
}
df = pd.DataFrame(data)

# --- 2. Parity Calculation ---
df['Bond (K*e^-rT)'] = K * np.exp(-r * T)
df['LHS (C + Bond)'] = df['Call Price (C)'] + df['Bond (K*e^-rT)']
df['RHS (P + S0)'] = df['Put Price (P)'] + S0
df['Parity Gap (LHS - RHS)'] = df['LHS (C + Bond)'] - df['RHS (P + S0)']

# The standard error of the gap is approximated by sqrt(SE_call^2 + SE_put^2)
# This assumes the estimators for C and P are uncorrelated, which is not strictly true
# but provides a reasonable benchmark for significance.
df['Gap SE (Approx.)'] = np.sqrt(df['Call SE']**2 + df['Put SE']**2)

# --- 3. Display Results ---
print("--- Q11: Put-Call Parity Check ---")
parity_results_df = df[['Model', 'LHS (C + Bond)', 'RHS (P + S0)', 'Parity Gap (LHS - RHS)', '
print(parity_results_df.round(4))
```

```
--- Q11: Put-Call Parity Check ---
              Model  LHS (C + Bond)  RHS (P + S0)  Parity Gap (LHS - RHS)  \
0  Heston (ρ=-0.30)         81.2732       83.0094                 -1.7362
1  Heston (ρ=-0.70)         80.0913       84.1131                 -4.0218
2   Merton (λ=0.75)         87.2168       87.1932                  0.0236
3   Merton (λ=0.25)         85.7931       85.7441                  0.0490

   Gap SE (Approx.)
0            0.0249
1            0.0247
2            0.0405
3            0.0392
```

Q12: Pricing Across Different Strikes

In [11]:

```
import numpy as np
import pandas as pd

# --- 1. Combined Model and Simulation Parameters ---
# General
S0 = 80.0
r = 0.055
T = 0.25

# Heston Specific
v0 = 0.032
kappa = 1.85
theta = 0.045
xi = 0.60
rho_heston = -0.30  # Base case for Heston

# Merton Specific
sigma_merton = 0.35
mu_j = -0.5
delta_j = 0.22
lam_merton = 0.25   # Base case for Merton

# MC Controls
N_SIMS = 60000
N_STEPS = 250
SEED = 42

# --- 2. Refined Pricing Functions ---

def price_heston_mc(s0_val, k_val, rho, seed):
    dt = T / N_STEPS
    sqrt_dt = np.sqrt(dt)
    rng = np.random.default_rng(seed)
    S = np.full(N_SIMS, s0_val, dtype=np.float64)
    v = np.full(N_SIMS, v0, dtype=np.float64)
    for _ in range(N_STEPS):
        z_v, z_s_ind = rng.standard_normal(size=(2, N_SIMS))
        z_s = rho * z_v + np.sqrt(1 - rho**2) * z_s_ind
```

```
        v_pos = np.maximum(v, 0)
        v = v + kappa * (theta - v_pos) * dt + xi * np.sqrt(v_pos) * sqrt_dt * z_v
        S = S * np.exp((r - 0.5 * np.maximum(v, 0)) * dt + np.sqrt(np.maximum(v, 0)) * sqrt_dt
    discount = np.exp(-r * T)
    call_payoff = np.maximum(S - k_val, 0)
    put_payoff = np.maximum(k_val - S, 0)
    return np.mean(call_payoff) * discount, np.mean(put_payoff) * discount

def price_merton_mc(s0_val, k_val, lam, seed):
    dt = T / N_STEPS
    sqrt_dt = np.sqrt(dt)
    rng = np.random.default_rng(seed)
    k_adj = np.exp(mu_j + 0.5 * delta_j**2) - 1.0
    drift = (r - lam * k_adj - 0.5 * sigma_merton**2) * dt
    log_S = np.log(s0_val) + np.sum(
        drift + sigma_merton * sqrt_dt * rng.standard_normal(size=(N_STEPS, N_SIMS)) +
        mu_j * rng.poisson(lam * dt, size=(N_STEPS, N_SIMS)) +
        delta_j * np.sqrt(rng.poisson(lam * dt, size=(N_STEPS, N_SIMS))) * rng.standard_normal
        axis=0
    )
    ST = np.exp(log_S)
    discount = np.exp(-r * T)
    call_payoff = np.maximum(ST - k_val, 0)
    put_payoff = np.maximum(k_val - ST, 0)
    return np.mean(call_payoff) * discount, np.mean(put_payoff) * discount

# --- 3. Main Execution Loop ---

moneyness_levels = [0.85, 0.90, 0.95, 1.00, 1.05, 1.10, 1.15]
strikes = [S0 / m for m in moneyness_levels]

results_list = []
for k in strikes:
    c_heston, p_heston = price_heston_mc(S0, k, rho=rho_heston, seed=SEED)
    c_merton, p_merton = price_merton_mc(S0, k, lam=lam_merton, seed=SEED)
    results_list.append({
        'Strike': k,
        'Moneyness (S0/K)': S0/k,
        'Heston Call Price': c_heston,
        'Heston Put Price': p_heston,
        'Merton Call Price': c_merton,
        'Merton Put Price': p_merton
    })

results_df = pd.DataFrame(results_list)

# --- 4. Display Results ---
print("--- Q12: Heston vs. Merton Pricing Across Strikes ---")
print(results_df.round(4))
```

```
--- Q12: Heston vs. Merton Pricing Across Strikes ---
    Strike  Moneyness (S0/K)  Heston Call Price  Heston Put Price  \
0  94.1176              0.85             0.0927           14.6613
1  88.8889              0.90             0.3110            9.7223
2  84.2105              0.95             0.9362            5.7330
3  80.0000              1.00             2.3657            3.0094
4  76.1905              1.05             4.6296            1.5159
5  72.7273              1.10             7.2962            0.7665
6  69.5652              1.15            10.0434            0.3949

   Merton Call Price  Merton Put Price
0             2.2482           14.9961
1             3.5414           11.1319
2             5.1719            8.1480
3             7.0902            5.9132
4             9.2155            4.2810
5            11.4594            3.1090
6            13.7483            2.2790
```

Step 2

Q14: European Up-and-In Call Option (Heston)

In [9]:
```python
import numpy as np

# Re-using the Heston pricing function from Team A, modified for a barrier option.
def price_heston_cui_mc(s0_val, k_val, rho, barrier, seed):
    dt = T / N_STEPS
    sqrt_dt = np.sqrt(dt)
    rng = np.random.default_rng(seed)

    S = np.full(N_SIMS, s0_val, dtype=np.float64)
    v = np.full(N_SIMS, v0, dtype=np.float64)
    max_S = np.copy(S) # Array to track the max price for each path

    for _ in range(N_STEPS):
        z_v = rng.standard_normal(N_SIMS)
        z_s_ind = rng.standard_normal(N_SIMS)
        z_s = rho * z_v + np.sqrt(1 - rho**2) * z_s_ind

        v_pos = np.maximum(v, 0)
        v = v + kappa * (theta - v_pos) * dt + xi * np.sqrt(v_pos) * sqrt_dt * z_v

        S = S * np.exp((r - 0.5 * np.maximum(v, 0)) * dt + np.sqrt(np.maximum(v, 0)) * sqrt_dt
        max_S = np.maximum(max_S, S) # Update the max price at each step

    # Payoff is conditional on the barrier being crossed
    call_payoffs = np.maximum(S - k_val, 0)
    knocked_in_indicator = (max_S >= barrier)
    cui_payoffs = call_payoffs * knocked_in_indicator

    price = np.mean(cui_payoffs) * np.exp(-r * T)
    return price

# Parameters from Q6 (ρ=-0.70)
B_cui = 95.0
K_cui = 95.0
price_cui = price_heston_cui_mc(S0, K_cui, rho=-0.70, barrier=B_cui, seed=42)
vanilla_call_q6 = 1.1838 # From Team A's results

print("--- Q14: Heston Up-and-In Call Option ---")
print(f"European Up-and-In Call Price (B=95, K=95): ${price_cui:.4f}")
print(f"Vanilla European Call Price (from Q6, K=80): ${vanilla_call_q6}")
```

```
--- Q14: Heston Up-and-In Call Option ---
European Up-and-In Call Price (B=95, K=95): $0.0029
Vanilla European Call Price (from Q6, K=80): $1.1838
```

Q15: European Down-and-In Put Option (Merton)

In [10]:
```python
import numpy as np

# --- 1. Define All Necessary Parameters ---
# General
S0 = 80.0
r = 0.055
T = 0.25
sigma = 0.35

# Merton
mu_j = -0.5
delta_j = 0.22

# MC Controls
N_SIMS = 20000
N_STEPS = 252

def price_merton_pdi_mc(s0_val, k_val, lam, barrier, seed):
    dt = T / N_STEPS
    sqrt_dt = np.sqrt(dt)
    rng = np.random.default_rng(seed)
```

```
    # Drift adjustment
    k = np.exp(mu_j + 0.5 * delta_j**2) - 1.0
    drift = (r - lam * k - 0.5 * sigma**2) * dt

    S = np.full(N_SIMS, s0_val, dtype=np.float64)
    min_S = np.copy(S)

    for _ in range(N_STEPS):
        z = rng.standard_normal(N_SIMS)
        z_j = rng.standard_normal(N_SIMS)
        poisson_jumps = rng.poisson(lam * dt, N_SIMS)

        jump_comp = mu_j * poisson_jumps + delta_j * np.sqrt(poisson_jumps) * z_j
        S = S * np.exp(drift + sigma * sqrt_dt * z + jump_comp)
        min_S = np.minimum(min_S, S)

    put_payoffs = np.maximum(k_val - S, 0)
    knocked_in_indicator = (min_S <= barrier)
    pdi_payoffs = put_payoffs * knocked_in_indicator

    price = np.mean(pdi_payoffs) * np.exp(-r * T)
    return price

# Parameters from Q8 (λ=0.75)
B_pdi = 65.0
K_pdi = 65.0
price_pdi = price_merton_pdi_mc(S0, K_pdi, lam=0.75, barrier=B_pdi, seed=8)
vanilla_put_q8 = 7.19 # From Team B's refined results

print("--- Q15: Merton Down-and-In Put Option ---")
print(f"European Down-and-In Put Price (B=65, K=65): ${price_pdi:.4f}")
print(f"Vanilla European Put Price (from Q8, K=80): ${vanilla_put_q8}")
```

```
--- Q15: Merton Down-and-In Put Option ---
European Down-and-In Put Price (B=65, K=65): $2.8172
Vanilla European Put Price (from Q8, K=80): $7.19
```